

DOCUKIDS – Guía Técnica y Plan de Construcción (v1.2)

Propósito de este documento: Explicar, con el máximo detalle pero sin tecnicismos innecesarios, **cómo vamos a construir Docukids**. Aquí encontrarás las herramientas que usaremos, cómo se comunican entre sí, dónde se guarda la información y qué hay que hacer, en qué orden. Esto es el **"manual de obras"** que complementa al croquis conceptual ([Docukids.md](#)).

Cambios respecto a v1.1 (esta revisión):

1. Se elimina **Railway** de las opciones de despliegue: dejó de ser gratis (exige tarjeta y cobra desde el primer mes). Se usa solo **Render**.
2. Se fija el modelo de Gemini a usar (**Flash / Flash-Lite**, no Pro) porque Pro dejó de ser gratuito.
3. Se documenta que **Supabase pausa proyectos gratuitos tras 7 días sin actividad**, y se explica por qué el cron diario que ya necesitábamos soluciona esto de forma gratuita, sin pasos extra.
4. Se corrige el contrato de `/upload`, que no pedía `name` ni `type` aunque la tabla `documents` los exige.
5. Se reescribe la sección de reparto de responsabilidades: el equipo nominal es de 4 personas pero en la práctica el proyecto lo construye **una sola persona con apoyo de IA**, así que se reemplazan los "roles" por un **plan de construcción secuencial**.
6. Se añade una nota sobre privacidad al usar el nivel gratuito de Gemini con documentos de menores.
7. Se añade al final una tabla de verificación de servicios gratuitos, fechada, para que quede constancia de que todo el stack es gratuito hoy.

1. ¿Qué herramientas vamos a usar y por qué?

Hemos elegido un stack moderno pero accesible, pensando en que sea fácil de aprender, **100% gratuito** y que funcione bien en un entorno universitario.

- **Para la aplicación móvil (Frontend):** Usaremos **React Native con Expo (SDK 52)**. Es el estándar actual para hacer apps que funcionen en Android sin necesidad de

instalar Android Studio pesado. Vamos a escribir todo en **TypeScript**, lo que nos ayuda a evitar errores tontos porque el código avisa si algo no cuadra.

- Para movernos entre pantallas, usamos **Expo Router**, que organiza las vistas como si fueran carpetas.
- La comunicación con el servidor la haremos con **Axios** (una librería para hacer peticiones HTTP).
- Para la cámara y la galería, usaremos **expo-image-picker**.
- El calendario visual dentro de la app lo montaremos con **react-native-calendars** (solo como vista, no escribe en el calendario del sistema).
- Para la autenticación, usaremos **@supabase/supabase-js** directamente desde la app.
- **Nota:** **react-native-calendar-events** y **expo-notifications** quedan fuera del MVP. No son necesarios: el calendario real es Google Calendar (lo escribe el backend) y los avisos van por correo, no por notificación push.
- **Para el servidor (Backend):** Usaremos **Python 3.10+** con el framework **FastAPI**. Es ligero, muy rápido y genera automáticamente una documentación interactiva (algo que nos salvará al hacer pruebas). La base de datos la manejaremos con **SQLAlchemy** (un ORM que traduce Python a SQL) y **asyncpg** para conectar con la base de datos.
- **Para la base de datos:** Usaremos **PostgreSQL alojado en Supabase** (plan gratuito). Supabase nos da dos cosas clave: la base de datos en sí y el sistema de autenticación (registro e inicio de sesión) ya resuelto, sin tener que programarlo desde cero.
 - **⚠ Importante (verificado sept. 2026):** los proyectos gratuitos de Supabase se pausan solos tras **7 días sin actividad** en la base de datos. No hace falta ningún truco extra para evitarlo: el cron diario del scheduler (ver más abajo) hace una consulta a la base de datos todos los días, así que **nunca pasan 7 días sin actividad** y el proyecto no se pausa. Es un efecto colateral positivo de tener el recordatorio automático — otra razón más para no saltarse el scheduler.
- **Para guardar los archivos (PDF y fotos):** No los guardaremos en nuestro servidor, porque pesan mucho. Usaremos **Google Drive**. **El backend será el encargado de subir los archivos** usando un **refresh_token** de Google asociado al usuario. El frontend nunca toca un token de Google.
- **Para leer las fechas de los documentos (OCR):** Usaremos la **API de Gemini** (obtenida desde **Google AI Studio** con una API Key gratuita, no desde Google Cloud Console). Le pasamos la imagen y ella nos devuelve el texto; nosotros buscaremos la fecha de vencimiento.
 - **⚠ Modelo a usar:** especifica en el código el modelo **gemini-2.0-flash-lite** o **gemini-flash** (verificado sept. 2026). Los modelos **Pro dejaron de ser gratuitos en 2026**; si el código pide un modelo Pro por error, la llamada va a fallar o a cobrar.
 - **⚠ Nota de privacidad (importante porque son documentos de menores):** en el nivel gratuito, Google puede usar el contenido enviado a la API para mejorar sus modelos. Como estamos mandando fotos de DNI,

carnets de vacunación, etc. de niños, conviene dejarlo dicho explícitamente en el informe del proyecto como una limitación conocida y aceptada por alcance académico (no hay presupuesto para el nivel de pago, que sí da esa garantía de privacidad).

- **Para el calendario de turnos:** Usaremos **Google Calendar**. El backend escribirá los eventos usando el mismo `refresh_token` del usuario. **No usaremos el calendario nativo del teléfono** en el MVP: es Google Calendar, gestionado por el backend.
- **Para enviar correos de recordatorio:** Usaremos la **API de Gmail**. Así los avisos de "tu documento vence en 15 días" llegarán al correo. Esta es la función más importante del proyecto (según prioridad del equipo), así que cuídala especialmente al probarla.
- **Para el scheduler (recordatorios automáticos):** Usaremos un **cron externo (cron-job.org)** que llamará a un endpoint del backend una vez al día. **No usaremos APScheduler**, porque en el plan gratuito de Render el servidor se duerme y APScheduler no se ejecutaría estando dormido. Este cron cumple **dos funciones a la vez**: dispara la revisión de vencimientos Y evita que Supabase pause el proyecto (ver nota más arriba). **No es opcional — es el corazón del sistema de alertas**.
- **Para el despliegue (poner el servidor en internet):** Usaremos **Render.com**, en su plan gratuito (Hobby, \$0/mes). **Railway ya no tiene un plan realmente gratuito desde 2023: pide tarjeta y cobra un mínimo mensual, así que lo sacamos de las opciones**. En Render, si nadie usa la app durante 15 minutos, el servidor se "duerme" y la primera petición puede tardar entre 30 y 60 segundos en responder (el famoso **cold start**). Lo tenemos en cuenta con el cron externo, que lo despierta todos los días a la misma hora.

2. Las APIs externas que tendremos que configurar

Cada una de estas herramientas externas necesita credenciales (claves) para funcionar. Las guardaremos en un archivo `.env` que **nunca se sube al código**.

Vamos a necesitar configurar estos servicios:

- **Supabase (Auth y DB):** Nos dará una URL y una clave anónima (`SUPABASE_URL` y `SUPABASE_ANON_KEY`) para que el frontend se registre y haga login directamente. También nos dará una cadena de conexión (`DATABASE_URL`) para que nuestro backend hable directamente con la base de datos. Y una clave `SUPABASE_JWT_SECRET` (o el endpoint de validación) para que el backend verifique los tokens que le lleguen.
- **Google Cloud Console:** Aquí habilitaremos las APIs de **Drive**, **Calendar** y **Gmail**. Crearemos un cliente **OAuth de tipo "Aplicación web"** (no "Escritorio", no "Android"), porque el backend es quien hará el intercambio de código por tokens. La pantalla de consentimiento se pondrá en modo **"Externo"** y añadiremos los correos del equipo (hasta 100 usuarios de prueba, más que suficiente para un proyecto solo) como usuarios de prueba, así Google no nos pedirá revisión de seguridad. El

backend guardará el `refresh_token` de cada usuario para poder actuar en su nombre.

- ⚠ **Detalle que suele romper esto:** Google solo entrega un `refresh_token` la **primera vez** que un usuario autoriza la app, salvo que la URL de autorización incluya explícitamente `access_type=offline` y `prompt=consent`. Si te olvidás de estos dos parámetros al generar la URL en `/auth/google/url`, en la segunda prueba (o si el usuario revoca y vuelve a aceptar) puede que no recibas el `refresh_token` y el flujo se rompa en silencio. **Guárdalo como checklist para Backend 2 / para vos mismo.**
- **Google AI Studio (Gemini):** Aquí (no en Google Cloud Console) generaremos una **API Key** (`GEMINI_API_KEY`). Es gratis y suficiente para nuestro MVP siempre que se use un modelo **Flash** (ver sección 1).
- **cron-job.org:** Servicio gratuito que llamará a nuestro endpoint `/scheduler/check` todos los días a las 8:00 a.m., enviando una cabecera `X-Cron-Secret` con la clave que definamos en el backend.

Flujo OAuth de Google resumido

El frontend abre en el navegador una URL que genera el backend (`/auth/google/url`). El usuario autoriza. Google redirige a `/auth/google/callback` del backend. El backend intercambia el código por un `refresh_token` y lo guarda asociado al usuario. A partir de ahí, el backend puede subir a Drive y crear eventos en Calendar sin intervención del frontend.

3. El Esquema de la Base de Datos (lo que guardamos en Supabase)

Aquí solo guardamos **metadatos**, nunca los archivos pesados. Ejecuta este código SQL directamente en el Editor SQL de Supabase para crear las tablas. Es más fácil copiar y pegar esto que rellenar un formulario:

-- Tabla de usuarios (vinculada al Auth de Supabase)

```
CREATE TABLE users (  
  id UUID PRIMARY KEY, -- Este ID lo genera Supabase al registrarse  
  email TEXT UNIQUE NOT NULL,  
  full_name TEXT,  
  google_refresh_token TEXT, -- Token para actuar en nombre del usuario en  
  Drive/Calendar/Gmail  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- Tabla de hijos (cada hijo pertenece a un usuario)

```
CREATE TABLE children (  
  id SERIAL PRIMARY KEY,  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  name TEXT NOT NULL,  
  birth_date DATE,  
  avatar_url TEXT,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- Tabla de documentos (aquí solo guardamos la referencia a Drive y la fecha de vencimiento)

-- NOTA: no guardamos 'status' porque se calcula al vuelo según la fecha actual.

```
CREATE TABLE documents (  
  id SERIAL PRIMARY KEY,  
  child_id INTEGER REFERENCES children(id) ON DELETE CASCADE,  
  name TEXT NOT NULL,          -- Ej. "DNI", "Cartilla de Vacunación"  
  type TEXT,                   -- Categoría: "identidad", "salud"  
  expiry_date DATE,            -- La fecha que extrae el OCR  
  drive_file_id TEXT,          -- ID único del archivo en Google Drive  
  drive_link TEXT,             -- URL para ver/descargar el archivo  
  upload_date DATE DEFAULT CURRENT_DATE,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- Tabla de turnos médicos

-- 'calendar_event_id' es el ID del evento en GOOGLE CALENDAR (no en el calendario nativo)

```
CREATE TABLE appointments (  
  id SERIAL PRIMARY KEY,  
  child_id INTEGER REFERENCES children(id) ON DELETE CASCADE,  
  title TEXT NOT NULL,  
  date DATE NOT NULL,  
  time TIME,  
  notes TEXT,  
  calendar_event_id TEXT,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Sobre el **status de los documentos:** No se persiste. El backend lo calcula en cada consulta comparando **expiry_date** con la fecha actual:

- **vencido:** **expiry_date** < hoy

- **proximo:** `expiry_date` entre hoy y hoy + 30 días
- **vigente:** `expiry_date` > hoy + 30 días

4. El Contrato de la API (Endpoints que debe tener el Backend)

Aquí sí mantengo una tabla, porque para los desarrolladores es el mapa de carreteras. Indica qué dirección URL llamar, qué método usar y qué devuelve.

Regla general: Todos los endpoints que tienen "Sí" en la columna de autenticación requieren que envíes un token en la cabecera:

`Authorization: Bearer <token_obtenido_al_login>`

Nota importante: No existen endpoints `/auth/register` ni `/auth/login`. El registro y el login los hace el frontend directamente contra Supabase. El backend solo valida el JWT que Supabase emite.

Método	Endpoint	¿Autenticación?	¿Qué espera?	¿Qué devuelve? (en caso de éxito)
GET	<code>/auth/google/url</code>	Sí	Nada	<code>{ url }</code> (URL para que el usuario autorice Drive/Calendar/Gmail; el backend debe construirla con <code>access_type=offline&prompt=consent</code>)
GET	<code>/auth/google/callback</code>	No	Parámetros <code>code</code> y <code>state</code> de Google	Redirige al frontend con <code>?google=ok</code>
GET	<code>/children</code>	Sí	Nada en el cuerpo	<code>{ children: [{id, name, birth_date}] }</code>
POST	<code>/children</code>	Sí	<code>{ name, birth_date? }</code>	<code>{ child }</code> (el nuevo perfil creado)

POST	/upload	Sí	Envía multipart/form-data con <code>file</code> , <code>child_id</code> , <code>name</code> (ej. "DNI") y <code>type</code> (ej. "identidad", "salud", opcional)	<code>{ status, expiry_date, drive_link, doc_id }</code>
GET	/documents/{child_id}	Sí	El <code>child_id</code> va en la URL	<code>{ documents: [{id, name, expiry_date, status, drive_link}] }</code> (el <code>status</code> se calcula al vuelo)
DELETE	/documents/{doc_id}	Sí	El <code>doc_id</code> va en la URL	<code>{ message: "Deleted" }</code>
POST	/appointments	Sí	<code>{ child_id, title, date, time, notes? }</code>	<code>{ ok, calendar_event_id }</code> (evento creado en Google Calendar)
GET	/appointments/{child_id}	Sí	El <code>child_id</code> va en la URL	<code>{ appointments: [...] }</code>
POST	/scheduler/check	Clave interna	Cabecera <code>X-Cron-Secret</code>	<code>{ processed: N }</code>

🔧 **Corrección respecto a v1.1:** el contrato original de `/upload` solo pedía `file` y `child_id`, pero la tabla `documents` exige `name` (NOT NULL) y tiene una columna `type`. Sin este cambio, el endpoint no podría guardar el documento. Ahora `/upload` también recibe `name` (**obligatorio**) y `type` (**opcional**). En la práctica, en la pantalla de subida el usuario elige/escribe el nombre del documento antes o después de sacar la foto (ej. un selector simple: **"DNI", "Cartilla de Vacunación", "Pasaporte", "Otro"**).

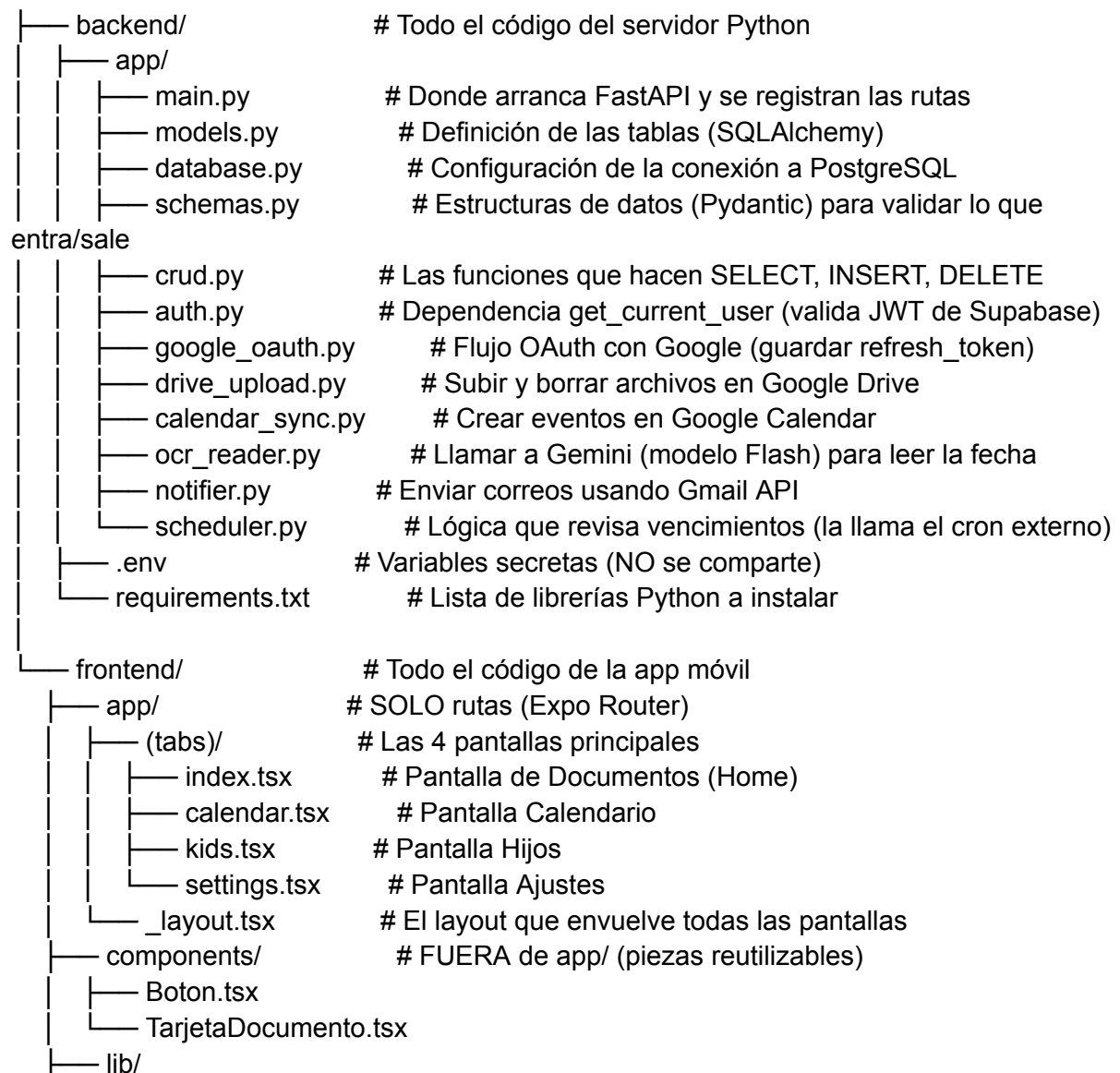
Sobre el endpoint `/scheduler/check`: Está pensado para que lo llame **cron-job.org** una vez al día. Requiere la cabecera **X-Cron-Secret** con el valor de la variable de entorno **CRON_SECRET**. Si la cabecera no coincide, devuelve **401**. Así nadie más puede dispararlo.

5. La Estructura de Carpetas (para no perdernos)

Tendremos dos grandes bloques: **backend/** y **frontend/**. Esta organización mantiene separado el servidor de la aplicación móvil.

Importante (Expo Router): en el frontend, la carpeta **components/** va **fuera de app/**. Todo archivo dentro de **app/** se convierte en una ruta, así que no queremos componentes sueltos generando rutas accidentales.

docukids/



```
| └─ supabase.ts      # Cliente de Supabase para el frontend
| └─ package.json
| └─ app.json         # Configuración de Expo
```

6. Cómo organizar el trabajo siendo una sola persona (con apoyo de IA)

El equipo figura como 4 integrantes, pero en la práctica el proyecto lo construye una persona sola, apoyándose en asistentes de IA para escribir y revisar código. Eso está bien — el stack elegido es manejable así — pero conviene ordenarse distinto a como se organizaría un equipo real, para no dispersarse.

- **No intentes avanzar en paralelo en varias partes.** Sin gente esperando tu entrega, lo que te va a hacer perder tiempo es saltar de tema en tema. Sigue el orden de la sección 7 (**Plan de construcción**) de arriba hacia abajo.
 - **Usa la IA para generar código de cada módulo, pero probá cada pieza antes de pasar a la siguiente.** Es más fácil detectar un error si acabás de tocar un solo archivo que si ya avanzaste tres pasos más.
 - **Guarda versiones que funcionan.** Aunque no uses Git, comprime la carpeta completa (`version_estable_1.0.zip`) cada vez que algo nuevo funcione de punta a punta. Si el paso siguiente rompe algo, volvés atrás sin perder lo ya probado. Si podés usar Git aunque sea básico (`git init`, `git add .`, `git commit -m "..."`), mejor: es gratis y te da el mismo resultado con menos fricción.
 - **Las claves (.env) son tuyas y de nadie más.** No hace falta compartir nada con terceros; igual conviene mantener un `env.example` con las variables vacías, documentando qué va en cada una, por si en algún momento alguien del equipo nominal quiere levantar el proyecto o para tu propia referencia dentro de unas semanas.
 - **Reserva tiempo aparte para el informe/documentación que pide la cátedra.** Al ser un proyecto solo, es fácil que quede para el final; ya tenés buena parte resuelta en este documento y en `Docukids.md`.
-

7. Plan de construcción (orden recomendado de tareas)

Este reemplaza la vieja "lista del Día 1": ahora es el mapa completo de construcción, en el orden que menos fricción genera al trabajar solo.

Fase 0 — Cuentas y credenciales (antes de escribir código)

1. Crea un proyecto gratuito en **Supabase**. En el Editor SQL, pega el script de la sección 3.

2. En **Google Cloud Console**, habilita las APIs de Drive, Calendar y Gmail. Crea credenciales OAuth de tipo "**Aplicación web**", pantalla de consentimiento en modo "**Externo**", y agrégate como usuario de prueba.

3. En **Google AI Studio**, genera la API Key de Gemini (**GEMINI_API_KEY**).

4. Crea una cuenta gratuita en **cron-job.org** (lo configuras al final, en la Fase 3).

5. Crea una cuenta gratuita en **Render** (lo usas en la Fase 4).

Fase 1 — Backend base (CRUD sin integraciones externas)

1. Crea el entorno virtual (`python -m venv venv`) y actívalo.

2. Instala dependencias:

```
pip install fastapi uvicorn sqlalchemy asyncpg python-dotenv supabase \
    google-auth google-auth-oauthlib google-api-python-client \
    google-generativeai httpx python-multipart
```

Guarda esta lista en `requirements.txt`. **No instales APScheduler.**

3. Implementa `models.py`, `database.py`, `schemas.py`, `auth.py` (validación del JWT de Supabase) y los endpoints de `/children`. Prueba que `uvicorn app.main:app --reload` levante sin errores y que la documentación automática (`/docs`) muestre los endpoints.

4. Implementa `/upload` como **simulado** (guarda el `name/type/child_id` en la base, sin tocar Drive todavía) y `/documents/{child_id}` con el `status` calculado al vuelo.

Fase 2 — Integraciones externas (una por una, probando cada una antes de seguir)

1. `google_oauth.py`: el flujo completo de `/auth/google/url` → `/auth/google/callback`, guardando el `refresh_token` (recordá `access_type=offline&prompt=consent`).

2. `drive_upload.py`: conecta `/upload` a la subida real en Drive.

3. `ocr_reader.py`: llama a Gemini (modelo Flash) para extraer la fecha de vencimiento del documento subido.

4. `calendar_sync.py`: implementa `/appointments` creando el evento real en Google Calendar.

5. `notifier.py`: implementa el envío de correos con Gmail API (o `smtplib` con una contraseña de aplicación de Gmail si la API te complica el setup — es una alternativa válida y también gratuita).

6. `scheduler.py`: la lógica que revisa vencimientos y llama a `notifier.py`, expuesta en `/scheduler/check` protegida con `X-Cron-Secret`.

Fase 3 — Automatización

1. Configura **cron-job.org** para llamar a `/scheduler/check` una vez al día (8:00 a.m.), con la cabecera `X-Cron-Secret`.

2. Verifica manualmente (llamando al endpoint con Postman o `curl`) que el correo llega antes de confiar en que el cron lo hará solo.

Fase 4 — Despliegue del backend

1. Sube el backend a **Render** (plan gratuito). Configura las variables de entorno del `.env` en el panel de Render, no subas el archivo `.env` al repositorio.

2. Confirma que el cron de la Fase 3 apunte a la URL pública de Render, no a `localhost`.

Fase 5 — Frontend

1. `npm install` (o `yarn install`) en `frontend/`, y `npx expo install` para las librerías de Expo de la sección 1. Incluye `@supabase/supabase-js`.

2. Maqueta las 4 pantallas (**Docs, Calendario, Hijos, Ajustes**) siguiendo los mockups de `Docukids.md`, con datos de prueba fijos ("hardcodeados") primero, sin conectar a la API todavía.

3. Conecta el registro/login contra Supabase, el interceptor de Axios con el header `Authorization`, la subida de archivos con `expo-image-picker` + `POST /upload`, el botón **"Conectar con Google"** (abre la URL de `/auth/google/url`), y el resto de las pantallas contra los endpoints reales.

Fase 6 — Empaquetado (APK)

1. Con el entorno de Android configurado (Android Studio, JDK), genera los archivos nativos:

```
npx expo prebuild --platform android
```

2. Compila el APK de release:

```
npx expo run:android --variant release
```

8. ¿Qué tenemos que entregar al final del proyecto?

El resultado final debe ser un producto que funcione. Asegúrate de tener esto listo:

- **Código fuente completo** (todo el backend y frontend) en una carpeta compartida o repositorio.
- **El backend funcionando en Internet** (una URL pública de Render).
- **Un archivo .apk** que podamos instalar en cualquier teléfono Android para la demostración. Para generarlo, recomendamos la construcción local (no depender de EAS Build), siguiendo estos pasos:

La Mejor Opción: Construir Localmente

Esta es la alternativa más directa y que no depende de ningún servicio externo. Puedes generar el APK directamente en tu propia computadora.

- **Cómo funciona:** Usas las herramientas nativas de Android (Gradle) para compilar la app. Expo facilita este proceso con un solo comando.
- **Pasos resumidos:**
 1. Genera los archivos nativos de Android:
`npx expo prebuild --platform android`
 2. Compila el APK de release:
`npx expo run:android --variant release`
Este comando generará un APK firmado que puedes instalar en cualquier dispositivo.
- **Ventajas:** Es completamente gratis, no tienes límites de uso y no dependes de la conexión a internet de EAS.
- **Desventaja:** Requiere tener el entorno de desarrollo de Android configurado (JDK, Android Studio, SDKs), lo cual ya es necesario para probar la app en un emulador.
- Un **README.md** sencillo que explique cómo instalar y ejecutar el proyecto localmente.
- (Opcional) Una colección de pruebas en Postman para enseñar que los endpoints responden correctamente.

MVP Priorizado (Funcionalidades vs. Simplificaciones)

Funcionalidad

¿Indispensable?

¿Puede simplificarse?

Registro/login con Supabase	✓ Sí	Directo desde el frontend. Sin endpoints propios en el backend.
Subida de documentos a Drive	✓ Sí	La hace el backend. Prueba primero con almacenamiento local (simulado) y luego integra Drive.
OCR con Gemini	✓ Sí	Imprescindible para la demo, pero ten un fallback (fecha manual) mientras pruebas. Usa modelo Flash , no Pro.
Listado de documentos por hijo	✓ Sí	Endpoint CRUD básico (GET, POST, DELETE). El status se calcula al vuelo.
Google Calendar (turnos)	✓ Sí	Lo gestiona el backend. Se elimina el calendario nativo del MVP.
Envío de correos con Gmail	✓ Sí — prioridad máxima	Puedes usar smtp lib con una cuenta Gmail (contraseña de aplicación) si la API te da problemas.
Scheduler automático (cron diario)	✓ Sí — no es opcional	Es lo que hace que el correo llegue solo, y de paso evita que Supabase pause el proyecto. No lo reemplaces por un botón manual: el equipo definió el envío automático como la función más importante.
Calendario nativo	✗ Fuera	Eliminado del MVP. Solo Google Calendar.
Notificaciones push	✗ Fuera	Eliminadas del MVP. Solo correos.

Regla de oro: el **Drive + OCR + Google Calendar + correo automático diario** es el núcleo que hay que mostrar funcionando de punta a punta. Todo lo demás (Postman, README elaborado) es secundario si el tiempo aprieta.

Resumen de decisiones clave (para consulta rápida)

Punto	Decisión
Subida a Drive	La hace el backend , no la app.

OCR con Gemini	El backend llama a Gemini (modelo Flash/Flash-Lite , no Pro) tras recibir el archivo. API Key de Google AI Studio .
Calendario	Solo Google Calendar , gestionado por el backend. Se elimina el calendario nativo.
Scheduler	Cron externo (cron-job.org), obligatorio, corre a diario. Se elimina APScheduler.
Autenticación	Supabase Auth directo desde el frontend . El backend solo valida el JWT.
Notificaciones push	Fuera del MVP . Solo correos con Gmail.
OAuth de Google	Cliente tipo "Aplicación web" . El backend guarda el refresh_token ; la URL de autorización debe llevar access_type=offline&prompt=consent .
Despliegue del backend	Render (gratuito). Railway se descarta: ya no tiene free tier real.
Estructura frontend	components/ fuera de app/ .
Columna status	No se persiste ; se calcula al vuelo.
Protección del scheduler	Cabecera X-Cron-Secret .
Contrato de /upload	Recibe también name (obligatorio) y type (opcional), no solo file y child_id .
Equipo	Nominalmente 4 personas; en la práctica 1 persona + IA. Sección 6/7 reescritas para ese caso real.

Verificación de servicios gratuitos (a septiembre de 2026)

Esta tabla existe para que quede constancia, en caso de que alguien del equipo o el profesor pregunte "**¿esto sigue siendo gratis?**". Verificado antes de cerrar esta revisión:

Servicio	¿Gratis hoy?	Detalle a tener en cuenta
----------	--------------	---------------------------

Render (backend)	✓ Sí, sin tarjeta	Plan Hobby \$0/mes. Se duerme tras 15 min sin uso; cold start de 30-60s. 750 h/mes incluidas.
Railway	✗ No	Descartado del stack. Desde 2023 exige tarjeta y cobra un mínimo mensual tras el trial de 30 días.
Supabase (DB + Auth)	✓ Sí, sin tarjeta	Proyecto se pausa tras 7 días sin actividad (mitigado por el cron diario del scheduler). Límite de 500 MB de base de datos, más que suficiente para el MVP.
Google AI Studio (Gemini)	✓ Sí, sin tarjeta	Solo modelos Flash/Flash-Lite ; Pro pasó a ser de pago en 2026. Datos del nivel gratuito pueden usarse para mejorar los modelos de Google (ver nota de privacidad en sección 1).
Google Cloud (Drive/Calendar/Gmail APIs)	✓ Sí, sin tarjeta	Gratis en modo "Externo / usuarios de prueba" (hasta 100 usuarios), que alcanza de sobra para un proyecto universitario.
cron-job.org	✓ Sí, sin tarjeta	Límite generoso de tareas gratuitas, más que suficiente para un ping diario.

Conclusión

Con las correcciones de esta revisión (**sacar Railway, fijar Gemini Flash, y aprovechar el cron para evitar la pausa de Supabase**), todo el stack sigue siendo **100% gratuito y apto para nivel universitario**.